

JavaScript Projects

Build a Portfolio of Real JS Applications

■ JS PROJECTS

■ LEARNING OUTCOME

By the end of this module you will have 4 complete, deployable JavaScript projects ready for your portfolio -- with localStorage, real API integration, and clean modular code.

■ Skills You Will Gain

- Build CRUD Todo App with localStorage persistence
- Create Weather App with real OpenWeatherMap API
- Build interactive Quiz App with timer and scoring
- Create Notes App with search and sorting
- Organise code with ES6 modules
- Deploy all projects on GitHub Pages

■ Projects are proof. Every JS interview includes a live coding challenge or portfolio review. Four well-built projects show more competence than any certificate.

SECTION 1

Project 1 -- Todo App with localStorage

```
const state = { todos: JSON.parse(localStorage.getItem('todos')) || [], filter: 'all', nextId:
parseInt(localStorage.getItem('nextId')) || 1 }; function addTodo(text) { if (!text.trim()) return;
state.todos.unshift({ id:state.nextId++, text:text.trim(), completed:false, createdAt:new
Date().toISOString() }); saveAndRender(); } function toggleTodo(id) { const todo =
state.todos.find(t => t.id===id); if (todo) { todo.completed = !todo.completed; saveAndRender(); }
} function deleteTodo(id) { state.todos = state.todos.filter(t => t.id!==id); saveAndRender(); }
function saveAndRender() { localStorage.setItem('todos', JSON.stringify(state.todos));
localStorage.setItem('nextId', state.nextId); render(); } function getFiltered() {
switch(state.filter) { case 'active': return state.todos.filter(t=>!t.completed); case 'completed':
return state.todos.filter(t=>t.completed); default: return state.todos; } } // EVENT DELEGATION
document.querySelector('#list').addEventListener('click', (e) => { if (e.target.matches('.delete'))
deleteTodo(Number(e.target.dataset.id)); if (e.target.matches('.toggle'))
toggleTodo(Number(e.target.dataset.id)); });
document.querySelector('#add-form').addEventListener('submit', (e) => { e.preventDefault(); const
input = e.target.querySelector('input'); addTodo(input.value); input.value=''; input.focus(); });
```

SECTION 2

Project 2 -- Weather App

```
// OpenWeatherMap API -- free at openweathermap.org const API_KEY = 'YOUR_KEY'; const BASE =
'https://api.openweathermap.org/data/2.5'; async function getWeather(city) { const res = await
fetch(`${BASE}/weather?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric`); if
(res.status === 404) throw new Error(`City '${city}' not found`); if (!res.ok) throw new Error(`API
error: ${res.status}`); return res.json(); } function renderWeather(data) {
document.querySelector('.city').textContent = `${data.name}, ${data.sys.country}`;
document.querySelector('.temp').textContent = `${Math.round(data.main.temp)}°C`;
document.querySelector('.desc').textContent = data.weather[0].description;
document.querySelector('.icon').src =
`https://openweathermap.org/img/wn/${data.weather[0].icon}@2x.png`;
document.querySelector('.humidity').textContent = `${data.main.humidity}%`;
document.querySelector('.wind').textContent = `${data.wind.speed} m/s`; } // Search with debounce
(don't call API on every keystroke) const debouncedSearch = debounce(async (query) => { if
(query.length < 2) return; try { showSpinner(); const weather = await getWeather(query);
renderWeather(weather); } catch(err) { showError(err.message); } finally { hideSpinner(); } },
500); document.querySelector('#city-input').addEventListener('input', e=>
debouncedSearch(e.target.value.trim()) );
```

SECTION 3

Project 3 -- Quiz App

```
// Open Trivia Database -- completely free, no API key needed! //
https://opentdb.com/api.php?amount=10&category=18&difficulty=medium&type=multiple async function
fetchQuestions(amount=10, category=18, difficulty='medium') { const url = `https://opentdb.com/api.
php?amount=${amount}&category=${category}&difficulty=${difficulty}&type=multiple`; const { results
} = await (await fetch(url)).json(); return results.map(q => ({ question: decode(q.question),
correct: decode(q.correct_answer), options:
shuffle([q.correct_answer,...q.incorrect_answers]).map(decode) })); } // Decode HTML entities from
API function decode(str) { const t = document.createElement('textarea'); t.innerHTML = str; return
t.value; } // Fisher-Yates shuffle function shuffle(arr) { const a=[...arr]; for(let
i=a.length-1;i>0;i--) { const j=Math.floor(Math.random()*(i+1)); [a[i],a[j]]=a[j],a[i]]; } return
a; } const quiz = { questions:[], index:0, score:0, timerId:null, timeLeft:30, async start() {
this.questions=await fetchQuestions(); this.index=0; this.score=0; this.showQuestion(); },
```

```

showQuestion() { const q = this.questions[this.index];
document.querySelector('.question').textContent = q.question; const opts =
document.querySelector('.options'); opts.innerHTML=''; q.options.forEach(opt => { const
btn=document.createElement('button'); btn.textContent=opt;
btn.addEventListener('click',()=>this.check(opt)); opts.appendChild(btn); }); this.startTimer(); },
check(selected) { clearInterval(this.timerId); const q=this.questions[this.index];
if(selected===q.correct) this.score++; document.querySelectorAll('.options button').forEach(b=>{
b.disabled=true; if(b.textContent===q.correct) b.classList.add('correct'); else
if(b.textContent===selected) b.classList.add('wrong'); }); setTimeout(()=>{ this.index++;
if(this.index<this.questions.length) this.showQuestion(); else this.showResults(); },1500); },
startTimer() { this.timeLeft=30; clearInterval(this.timerId); this.timerId=setInterval(()=>{
document.querySelector('.timer').textContent--this.timeLeft;
if(this.timeLeft<=0){clearInterval(this.timerId);this.check('');} },1000); } }; quiz.start();

```

SECTION 4

Deployment & Interview Prep

1	File Structure Each project in own folder: /todo-app/, /weather-app/, /quiz-app/. Shared utilities in /shared/utlis.js.
2	Modular Code One concern per file: api.js, storage.js, render.js, events.js. Import with ES6 modules.
3	README per Project Problem statement, technologies, features, live demo link, how to run, what you learned.
4	GitHub Repository Public repo with meaningful commits: 'feat: add search filter', 'fix: debounce API calls'.
5	GitHub Pages Settings > Pages > Deploy from main. URL: username.github.io/todo-app
6	Code Quality ESLint + Prettier, no console.log in production, comments for complex logic.
7	Lighthouse Score Aim for 90+ on Performance, Accessibility, Best Practices, SEO.
8	Interview Prep Practice: implement debounce from scratch, deep clone, Promise.all from scratch.

Interview Topic	What to Prepare
Closures	Code a counter factory and memoization function
Event Loop	Trace output of async+sync+setTimeout code
'this' keyword	What 'this' is in: method, arrow fn, callback, class
Debounce	Code debounce from scratch -- common interview question
Deep Clone	Implement without JSON.stringify
Promises	Implement Promise.all from scratch
Array methods	Implement map, filter, reduce using for loops
Async/await	Convert Promise chain to async/await and back

■ PRACTICE Q & A

Q1. What JS feature persists data across page reloads?

A: localStorage -- stores string data permanently in browser. Use JSON.stringify to save, JSON.parse to load. sessionStorage for tab-only persistence. IndexedDB for large structured data.

Q2. How do you prevent XSS when rendering user text in DOM?

A: Use element.textContent = userText instead of innerHTML. textContent treats string as plain text -- HTML tags shown literally. innerHTML parses and can execute malicious scripts.

Q3. How would you implement search without API call on every keystroke?

A: Debounce: wait 300ms after user stops typing before calling API. clearTimeout each keystroke, setTimeout in handler. Prevents hundreds of requests.

Q4. What is the difference between localStorage, sessionStorage, and cookies?

A: localStorage: persistent, no expiry, 5-10MB. sessionStorage: cleared when tab closes. cookies: sent with every HTTP request, 4KB, server-accessible. Use localStorage for client-only data.

Q5. How do you structure a large JS project?

A: ES6 modules: api.js (fetch), storage.js (localStorage), components.js (DOM builders), events.js (listeners), utils.js (helpers). Bundler (Vite) for production build.

■ JS Projects Cheat Sheet	
<code>localStorage.setItem(k,v)</code>	Save string persistently
<code>JSON.stringify(obj)</code>	Object to string for storage
<code>JSON.parse(str)</code>	String to object
<code>crypto.randomUUID()</code>	Generate unique ID
<code>element.textContent=t</code>	Safe text -- no XSS
<code>createDocumentFragment()</code>	Batch DOM updates
<code>new URLSearchParams(obj)</code>	Build query string
<code>encodeURIComponent(str)</code>	Encode URL parameter
<code>debounce(fn,300)</code>	Delay API calls
<code>new FormData()</code>	Form/file data
<code>new Blob([data],[type])</code>	Create download file
<code>Fisher-Yates shuffle</code>	<code>arr[i],[arr[j]]=[arr[j],arr[i]]</code>